

Efficient Volumetric PolyCube-Map Construction

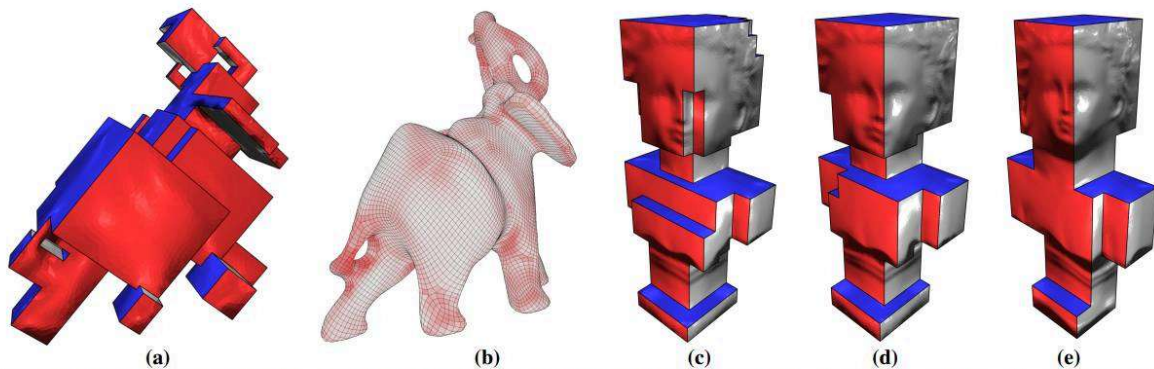
Xiao-Ming Fu^{1,2}Chong-Yang Bai^{1,2}Yang Liu²¹ University of Science and Technology of China, P. R. China² Microsoft Research Asia, P. R. China

Figure 1: (a) The volumetric PolyCube-map for the Elephant model (48k tetrahedrons). Our method took 17 seconds to achieve the PolyCube-map whose maximum isometric distortion is 11.7 and the number of corners is 176. (b) The all-hex meshing result from (a). The hexahedral mesh has 46994 hexes, and the minimal and average scaled Jacobian are 0.231 and 0.887. The hexes are color-coded by their scaled Jacobian values, with white being optimal. PolyCube-maps for the Buste model using different Gaussian smoothing kernel σ are shown in (c), (d) and (e). From (c) to (e), the results have 128 ($\sigma = \bar{l}_e$ where $\bar{l}_e$ is the average boundary edge length of the input mesh), 72 ($\sigma = 1.5\bar{l}_e$) and 36 ($\sigma = 2\bar{l}_e$) corners and the average isometric distortions are 1.50, 1.53 and 1.57, respectively.

Abstract

PolyCubes provide compact representations for closed complex shapes and are essential to many computer graphics applications. Existing automatic PolyCube construction methods usually suffer from poor quality or time-consuming computation. In this paper, we provide a highly efficient method to compute volumetric PolyCube-maps. Given an input tetrahedral mesh, we utilize two novel normal-driven volumetric deformation schemes and a polycube-allowable mesh segmentation to drive the input to a volumetric PolyCube structure. Our method can robustly generate foldover-free and low-distortion PolyCube-maps in practice, and provide a flexible control on the number of corners of Polycubes. Compared with state-of-the-art methods, our method is at least one order of magnitude faster and has better mapping qualities. We demonstrate the efficiency and efficacy of our method in PolyCube construction and all-hexahedral meshing on various complex models.

Categories and Subject Descriptors (according to ACM CCS): I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling—Geometric algorithms, languages, and systems

1. Introduction

PolyCubes extracted from 3D volumes are proved to be very useful to many computer graphics applications, such as texture mapping [THCM04, YL08, CL10], all-hexahedral meshing [GSZ11, HJS*14, FXBH16], and GPU-based subdivision [XGH*11]. A high-quality PolyCube-map construction algorithm usually satisfies the following properties in our view: (1) the algorithm is automatic without or with minimal manual intervention; (2) the PolyCube-map is free of foldovers and degenerate elements; (3) the mapping distortion is as low as possible; (4) the complexity of the PolyCube, i.e.

the number of corners, is controllable; (5) the algorithm should be efficient, otherwise manual construction is preferable.

From manual creation to automatic generation, there exist many attempts to obtain high-quality PolyCubes and PolyCube-maps. A typical algorithm [GSZ11] is based on mesh segmentation and deformation. The boundary surface of the input volumetric mesh is first segmented into separated charts whose orientation corresponds to the axis-aligned directions; then the mesh is deformed to match the pre-defined orientations for all the charts so that a PolyCube can be obtained by deformation. Livesu *et al.* [LVS*13] further improve

this method by adopting a graph-cut inspired approach to assign appropriate facet orientations. These two methods are sensitive to the initial orientation of the input mesh, and the elimination of turning points (that are also called *extrema*, *non-monotones*) is heuristic and time-consuming. Huang *et al.* [HJS*14] propose an l_1 -based optimization approach that is not sensitive to the initial orientation and provides a flexible control of the complexity of the result. However, the computational cost is high due to the heavy use of the sequential quadratic programming technique. The above methods also cannot guarantee that the mapping is free of foldovers.

We propose a highly efficient method to compute high-quality PolyCube-maps and overcome the above problems. Our method consists of four components:

1. A foldover-free normal-smoothing mesh deformation scheme that deforms the input tetrahedral mesh to a PolyCube-like shape. Particularly we introduce a novel normal smoothing operator to control the complexity of the results.
2. A simple mesh segmentation algorithm that determines whether the boundary surface is sufficient to form a valid PolyCube.
3. A foldover-free and normal-alignment mesh deformation that aligns the normals of boundary facets to axis directions. Combining with the normal-smoothing, the issues found in mesh segmentation can be resolved in practice.
4. A practically robust mesh flattening algorithm that yields the foldover-free and low-distortion PolyCube-map.

In the normal-smoothing mesh deformation and the normal-alignment mesh deformation, the best orientation of the input is computed so that the result of our algorithm is not affected by the initial orientation of the input. Our method can produce high-quality PolyCube-maps for complex shapes very efficiently. Compared with the existing deformation-based or optimization-based methods, our method is at least one order magnitude faster. We demonstrate the practical robustness of our method on various complex models (more than one hundred models) and show the superiority to existing methods in terms of both result quality and computational efficiency.

2. Related Work

Automatic PolyCube construction Manual construction of PolyCubes [THCM04] is a time-consuming task, especially for complex objects. Grid or Octree decomposition can generate PolyCubes naturally but the results are overly dense, and the corresponding mapping is distorted severely near the boundary. Lin *et al.* [LJFW08] construct PolyCubes by mesh segmentation and box-primitive fitting. The main issue of their method is the overly coarse results which may introduce high distortion. He *et al.* [HWFQ09] progressively construct a PolyCube structure via a scan-line strategy on critical points computed from a harmonic function. Their results may be over-refined on simple objects, and their volumetric mapping is computed separately and may contain large distortion. Gregson *et al.* [GSZ11] employ mesh deformation to reshape the volumetric input into a PolyCube structure, including two steps: the rotation-driven deformation aligns most surface normals to the axis directions; the position-driven deformation flattens the segmented charts to obtain the PolyCube result. Although their method usually results in a low-distortion PolyCube, it lacks the control of complexity (i.e., singularity counts) and the solution to eliminate turning

points is heuristic. Livesu *et al.* [LVS*13] overcome these issues by proposing a graph-cut based approach, but their algorithm could be time-consuming due to its local and greedy search. Recently Sokolov and Ray [SR15] tried to resolve the turning point issue by creating appropriate *steps*. Their unpublished approach looks promising, but its robustness requires an extensive test. Huang *et al.* [HJS*14] remove the orientation constraints of previous work through a minimization of the l_1 norm of mesh surface normals and also control flexibility by adjusting the similarity of normals between neighboring facets. They claim that they never encounter any turning points in practice in their high-quality results. However, their nonlinear solver is very time-consuming, especially on large-size models. Yu *et al.* [YZWL14] voxelize the result of rotational deformation [GSZ11] and apply a series of homotopic morphological operations to optimize the PolyCube structure. In this way, they avoid eliminating the turning points and find the PolyCube structure from the voxelization directly. However, their results are still overly dense compared to [LVS*13, HJS*14], and their volumetric mapping results are also computed separately and contain large distortions even foldovers. In contrast, our method can compute the PolyCube structure and the foldover-free volumetric mapping simultaneously in a very efficient way.

Volumetric PolyCube mapping A one-to-one mapping or locally injective mapping from the volumetric mesh to the PolyCube is essential to many applications, like hexahedral meshing [HXH10, GSZ11] and volumetric parameterization. Various numerical techniques have been developed for computing such a mapping. Methods like mean-value coordinates [Flo03], harmonic function [LGW*07] are popular but they have no guarantee to generate the bijective mapping. Recently, the locally injective mapping methods [SKPSH13] can find the volumetric mapping by minimizing the deformation distortion while preventing foldovers. However, for fixed boundary mappings, like PolyCubes, they usually fail because of too many positional constraints on the boundary. Bounded distortion mappings [AL13, SABL14] are well-designed optimization methods which can bound the mapping distortion such as conformal distortion and isometric distortion while ensuring no foldover in the results. However, the computational cost of these methods is relatively high due to the heavy use of quadratic programming or semidefinite programming, and it is hard to apply them to large-scale meshes. Kovalsky *et al.* [KABL15] improve these methods by proposing a first-order method and gain a significant speedup. For PolyCube-based mappings, the above methods assume that the PolyCube is given, and the distortion from the PolyCube construction is not considered. The deformation-based methods [GSZ11] find the PolyCube and the volumetric mapping simultaneously because the input tetrahedral mesh is deformed to a PolyCube. However, their method may result in foldovers or degenerate tetrahedrons because their deformation energy does not penalize the occurrence of these cases. Postprocessing is required to resolve these issues. Huang *et al.* [HJS*14] add the boundary foldover-free constraints in their objective function to prevent foldovers but require edge-collapse to handle possible degenerate tetrahedrons during the optimization and may fail to handle thin regions. We utilize the AMIPS energy [FLG15] in our method that can yield a low-distortion mapping without foldovers.

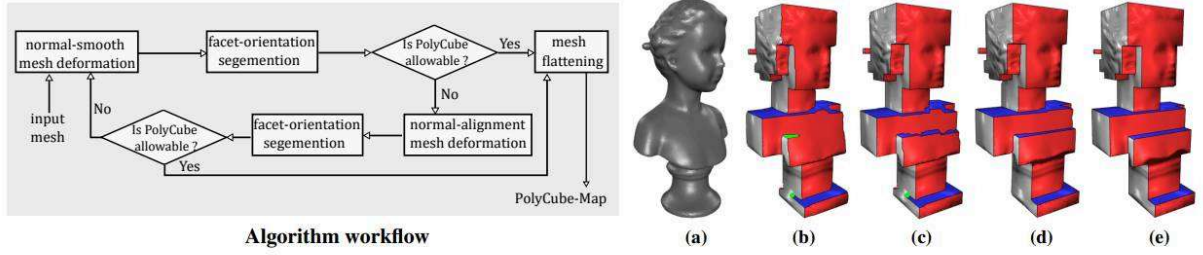


Figure 2: The left figure is our algorithm workflow. The right figure shows the results of each step of the algorithm applied to the Buste model. (a): The input tetrahedral mesh. (b): The result after the normal-smooth mesh deformation and mesh segmentation. Facets with the same label are with the same color. The segmentation result violates the sufficient topological conditions of the PolyCube structure. See the green chart that has only two neighboring charts. There is a turning point at the base (see the green circle). (c): The result after applying the normal-alignment deformation and mesh segmentation to (b). (d): The result after applying another round of the normal-smooth deformation and mesh segmentation to (c). The sufficient conditions of the PolyCube structure are satisfied and all the turning points are eliminated. (e): the final PolyCube-map after mesh flattening.

3. Algorithm

3.1. Overview

We design an efficient algorithm to compute the PolyCube-map for a given tetrahedral mesh. For a surface mesh input, we tetrahedralize it by TetGen [Si15] to obtain the tetrahedral mesh. We pre-process the tetrahedral input by splitting any interior edge or interior facet whose end points are all on the boundary surface just in case that these end points after the mapping are colinear and cause degenerate tetrahedrons. We propose a foldover-free volumetric deformation scheme, called *normal-smooth mesh deformation*, to deform the input so that the boundary facet normals are aligned to the projected Gaussian-smoothed normal directions as much as possible (Section 3.2). We apply a simple surface segmentation scheme to the deformation result to determine whether there exists a valid PolyCube topology (Section 3.3). If it exists, we use our mesh flattening method to find the PolyCube-map (Section 3.5). If the validity is violated, we apply a normal-alignment mesh deformation scheme to resolve the segmentation issues as many as possible (Section 3.4). The normal-alignment and normal-smooth deformation schemes are executed alternately until the sufficient conditions for PolyCube are satisfied and all the turning points are eliminated, or the maximal iteration number of alternating deformation is reached. The workflow of our algorithm is shown in Figure 2.

3.2. Normal-smooth mesh deformation

Denote the input mesh as $\mathcal{M}$ consisting of N tetrahedrons $\{\mathbf{t}_1, \dots, \mathbf{t}_N\}$, N_v vertices $\{\mathbf{v}_1^0, \dots, \mathbf{v}_{N_v}^0\}$. The facet set of its boundary triangle mesh $\mathcal{S}$ is denoted as $\mathcal{F} = \{\mathbf{f}_1, \dots, \mathbf{f}_{N_f}\}$ and the facet normals are $\{\mathbf{n}_1, \dots, \mathbf{n}_{N_f}\}$. We introduce an operator $\mathbb{A}(\cdot)$ that projects a 3D vector to its closest axis direction $((\pm 1, 0, 0)^T, (0, \pm 1, 0)^T$ or $(0, 0, \pm 1)^T$), for instance, $\mathbb{A}((3, 2, 1)^T) = (1, 0, 0)^T$. We know that if all the facet normals are perfectly aligned to one of axis directions, i.e. $\mathbf{n}_i = \mathbb{A}(\mathbf{n}_i), \forall i$, the boundary surface forms a PolyCube structure. The closest normal directions $\mathbb{A}(\mathbf{n}_i)$ have been used to guide the mesh deformation and segment the boundary surface into PolyCube charts [GSZ11]. However, it is also known that the direct normal assignment is too greedy, and usually introduces inconsistent and nonsmooth assignments. So instead of using $\mathbb{A}(\mathbf{n}_i)$ directly, we propose to combine the neighboring normals to guide the deformation.

For each boundary facet, we define a *normal-smooth* energy $E_s(\mathbf{f}_i)$:

$$E_s(\mathbf{f}_i) = \|\mathbf{n}_i - \mathbb{A}(G_\sigma(\mathbf{n}_i))\|^2.$$

$G_\sigma(\cdot)$ is a Gaussian function with the kernel size σ that averages the neighboring facet normals:

$$G_\sigma(\mathbf{n}_i) = \sum_{\mathbf{f}_j \in \mathcal{F}} \text{area}(\mathbf{f}_j) \exp\left(-\frac{\|\mathbf{q}_i - \mathbf{q}_j\|}{2\sigma^2}\right) \cdot \mathbf{n}_j,$$

and $\mathbf{q}_j$ is the center of $\mathbf{f}_j$. The weighted summation of $E_s(\mathbf{f}_i)$ defines the total normal-smooth energy

$$E_s := \sum_{\mathbf{f}_i} \mu_i E_s(\mathbf{f}_i),$$

here $\mu_i > 0$ is the weight of $\mathbf{f}_i$. We use E_s to guide the surface deformation. The kernel size σ plays an important role in controlling the complexity of the final PolyCube, a large value would reduce the complexity since it has a large smoothing effect on normals.

Note that E_s involves the boundary facets only. To achieve a foldover-free volumetric mapping, we need to move interior vertices. We employ the AMIPS method [FLG15] to achieve this goal. Denote matrix $\mathbf{A}_i$ as the linear transformation from the input tetrahedra $\mathbf{t}_i : \Delta \mathbf{v}_p^0 \mathbf{v}_q^0 \mathbf{v}_r^0 \mathbf{v}_s^0$ to its deformed version $\Delta \mathbf{v}_p \mathbf{v}_q \mathbf{v}_r \mathbf{v}_s$:

$$\mathbf{A}_i = [\mathbf{v}_p - \mathbf{v}_q \mid \mathbf{v}_p - \mathbf{v}_r \mid \mathbf{v}_p - \mathbf{v}_s] \cdot [\mathbf{v}_p^0 - \mathbf{v}_q^0 \mid \mathbf{v}_p^0 - \mathbf{v}_r^0 \mid \mathbf{v}_p^0 - \mathbf{v}_s^0]^{-1}.$$

The conformal and volumetric distortions on $\mathbf{t}_i$ are defined as:

$$\delta_i^{\text{conf}} = \frac{1}{8} \left(\|\mathbf{A}_i\|_F^2 \cdot \|\mathbf{A}_i^{-1}\|_F^2 - 1 \right),$$

$$\delta_i^{\text{vol}} = \frac{1}{2} \left(\det \mathbf{A}_i + \det \mathbf{A}_i^{-1} \right).$$

Here $\|\cdot\|_F$ is the Frobenius norm. By combining these two distortions, the *isometric AMIPS* energy on $\mathbf{t}_i$ is defined as:

$$E_{iso}(\mathbf{t}_i) = \exp\left(s \cdot \left(\alpha \delta_i^{\text{conf}} + (1 - \alpha) \delta_i^{\text{vol}}\right)\right).$$

The AMIPS energy goes to infinity if any deformed tetrahedra degenerates. Fu *et al.* [FLG15] show that the AMIPS energy can penalize the maximal distortion greatly and distribute the distortion evenly. We define $E_{iso} := \sum_{\mathbf{t}_i} E_{iso}(\mathbf{t}_i)$ as the total AMIPS energy. We set $s = 1$ and $\alpha = 0.5$ in our experiments.

We formulate the normal-smooth mesh deformation energy by combining the above energies.

$$E_{ns} := E_{iso} + E_s. \quad (1)$$

By minimizing E_{ns} with respect to vertex positions, the input volumetric mesh is deformed towards a PolyCube-like shape. However, it is obvious that the minimization result is affected by the global orientation of the input. Similar to [HJS*14], we need to determine the optimal global orientation first.

Optimal global orientation. We know that if the boundary surface can form a PolyCube, there exists a global rotation matrix $\mathbf{R}$ that can transform any boundary facet normal to one of the major axis directions, i.e., a coordinate entry of the rotated facet normal is 1 or -1. The following equation holds:

$$\Phi(\mathbf{Rn}_i) = 0, \forall \mathbf{f}_i.$$

Here $\mathbf{n}_i$ is the boundary facet normal of $\mathbf{f}_i$ and $\Phi(\mathbf{n}) := n_x^2 n_y^2 + n_y^2 n_z^2 + n_z^2 n_x^2$ where $\mathbf{n} = (n_x, n_y, n_z)^T$. For a general input, we can find the best global orientation $\mathbf{R}$ by minimizing $E_o := \sum_i \Phi(\mathbf{Rn}_i)$.

The minimization procedure of Eq. (1) is as follows. We seek the optimal rotation $\mathbf{R}$ and vertex positions alternately.

Update $\mathbf{R}$. We use the Euler angles to represent $\mathbf{R}$ and minimize E_o with respect to the Euler angles using the conjugate gradient method. Since there are only three variables and all the vertex positions are fixed, E_o can be minimized efficiently.

Update vertex positions. We rewrite $E_s(\mathbf{f}_i)$ as $\|\mathbf{Rn}_i - \mathbf{A}(G_\sigma(\mathbf{Rn}_i))\|^2$ and use the inexact block coordinate descent method [FLG15] to minimize E_{ns} :

1. Set the iteration number $k = 1$. Compute all the aligned Gaussian-smoothing facet normals $\mathbf{A}(G_\sigma(\mathbf{Rn}_i))$ and regard them as constants.
2. For each vertex $\mathbf{v}_i$, we fix other vertex positions and update its position by one-step gradient descent, i.e.,

$$\mathbf{v}_i^{new} := \mathbf{v}_i - \lambda \nabla_{\mathbf{v}_i} E_{ns}.$$

λ is the step size of the backtracking line search.

3. $k := k + 1$. If the maximum change of vertex positions is less than 10^{-4} or the maximum iteration number $k_{\max} = 100$ is reached, we stop the update. Otherwise, go to Step 2.

We notice that severely-distorted tetrahedrons should be avoided by putting more penalization to E_{iso} . However, when E_{iso} dominates E_{ns} , some $E_s(\mathbf{f}_i)$ s may deviate from zero largely due to the less penalization on them. Thus, it is necessary to dynamically adjust the weight of $E_s(\mathbf{f}_i)$ according to the current energy values to avoid the above issue. We propose the following μ_i -adjusting scheme to achieve this goal. For a boundary facet $\mathbf{f}_i$, denote its associated tetrahedra as $\mathbf{t}_i$. We use the current isometric distortion value of $\mathbf{t}_i$ and the normal-smooth energy on $\mathbf{f}_i$ to adjust μ_i at the l -th iteration of the alternating optimization of $\mathbf{R}$ and vertex positions:

$$\mu_i^l = \min(\max(\frac{E_{iso}(\mathbf{f}_i)}{E_s(\mathbf{f}_i)}, \mu_{\min}), \mu_{\max}).$$

Here $\mu_{\min} = 10^3$ and $\mu_{\max} = 10^{16}$ are the lower and upper bounds of μ_i used in our experiments.

Termination. The alternating optimization of $\mathbf{R}$ and vertex positions is stopped if the change of variables is smaller than 10^{-4} or the iteration number of the alternating optimization reaches its maximum (we set 10 in our experiment).

Speedup. Similar to the work of [FLG15], the vertex update procedure can be speeded up by partitioning the vertex set into multiple blocks via the graph coloring approach and parallelizing the vertex update for the vertices inside the same block.

3.3. Facet-orientation segmentation

After the normal-smooth deformation, most boundary parts of the mesh are nearly planar. We use the mesh segmentation approach [EM10, GSZ11, LVS*13] to check whether a PolyCube structure exists. We first label boundary facets as one of the directions $\{+X, -X, +Y, -Y, +Z, -Z\}$ according to $\mathbf{A}(\mathbf{Rn}_i)$, then group connected facets that share the same label to form facet clusters. These clusters are called *candidate PolyCube charts*. The edges between them are *candidate PolyCube edges* and the vertex sharing by at least three charts are *candidate PolyCube vertex*. We observe that some candidate PolyCube edges are very zig-zag and some candidate PolyCube charts may have not enough neighbors to form a valid PolyCube, we iteratively preprocess these candidate PolyCube edges and charts to get a refined segmentation result as follows.

1. **PolyCube-Edge straightening.** For any facet $\mathbf{f}$ adjacent to a PolyCube edge, if its two neighboring facets share the same label l_a and l_a is different from the label of $\mathbf{f}$, assign l_a to $\mathbf{f}$.
2. **PolyCube-Chart merging.** If a candidate PolyCube chart has less than four neighboring candidate PolyCube charts and its region is small (containing at most five boundary facets in our experiments), we set the labels of all the facets in this candidate chart as the label of one of neighboring candidate PolyCube facets.

Here merging small-region charts allows us to avoid introducing highly distorted charts and extra turning points. The order of chart merging has little effect on the final PolyCube layout according to our experience.

We then check the sufficient topological validity of the PolyCube structure by the following criterions [EM10]:

1. Any PolyCube facet should have at least four neighboring PolyCube facets.
2. Any two neighboring PolyCube facets should not have opposite labels such as $+X$ and $-X$.
3. The valence of each PolyCube vertex is three.

If the above criterions are satisfied, we check the turning points on the current candidate PolyCube structure by following the approach of [GSZ11]. For each candidate PolyCube edge, we first straighten it via smoothing its vertices by five steps of Laplacian smoothing, then project this smoothed PolyCube edge to the direction which is the cross-product of the orientations of its two neighboring candidate PolyCube charts. Turning points can be easily detected on the projected PolyCube edge as the vertices where the projected polyCube edge doubles back on itself. If no turning point is detected, we proceed the mesh flattening (Section 3.5) to compute the valid PolyCube map. In practice, some turning points may be falsely reported due to insufficient straightening. We can filter out

such turning points by measuring the severity of a turning point: the severity is defined by the area of the triangle that is formed by this turning point and its two nearest turning points or PolyCube vertices on the unsmoothed candidate PolyCube edge. If the severity of each turning point is less than three times of the average boundary facet area, we say that the severity level of turning points is low, and we can proceed mesh flattening. Although there is no theoretical guarantee that the falsely detected turning points are removed by our severity measurement, it works well in practice.

For the case that the sufficient topological validity is violated (for example, see the green chart in Figure 2b), or the severity of turning points (see the green circle in Figure 2b) is not low, highly non-planar candidate PolyCube charts usually exist. We apply the normal-alignment mesh deformation (Section 3.4) to improve the result.

3.4. Normal-alignment mesh deformation

To penalize highly non-planar candidate PolyCube charts, we introduce the *normal alignment energy* E_a to better align the normals to axis directions.

$$E_a = \sum_i \omega_i \Phi(\mathbf{Rn}_i).$$

Here ω_i is the weight of $\mathbf{f}_i$.

Similar to the normal-smooth mesh deformation, we also require that the distortion of the mapping is under control. We combine E_{iso} and E_a to form the normal-alignment energy:

$$E_{na} = E_{iso} + E_a.$$

The minimization procedure is similar to the normal-smooth deformation. The global orientation $\mathbf{R}$ and vertex positions are optimized alternately by the same updating schemes as the normal-smooth deformation. But the adjustment of ω_i is slightly different from the μ_i . Because we want to align the boundary facet normals to axis directions as much as possible, we require that E_a needs to dominate E_{na} gradually. Therefore, at each iteration of the alternating optimization, we adjust ω_i as follows:

$$\gamma^l = 10 \cdot \gamma^{l-1},$$

$$\omega_i^l = \min(\max(\gamma^l \cdot \frac{E_{iso}(\mathbf{f}_i)}{E_a(\mathbf{f}_i)}, \omega_{\min}), \omega_{\max}).$$

Here $\omega_{\min} = 10^3$, $\omega_{\max} = 10^{16}$ and γ^0 is set as 10^3 .

After the minimization, we run the facet-orientation segmentation to see whether the result is good enough for mesh flattening. If not, it is usually because some nearly-planar charts are isolated and they do not have enough number of neighboring charts. Figure 2c shows an example: the result has more planar charts than Figure 2b, but some charts in the same plane are not connected (see the middle region of Figure 2b). We switch to the normal-smooth mesh deformation to resolve this issue since the aligned Gaussian-smoothing normal operator encourages the neighboring regions to share similar normals, and the similar regions are more likely to be merged. Figure 2d shows the result after one pass of normal-smooth deformation + normal-alignment deformation + normal-smooth deformation. We observe that the violation of sufficient conditions of Polycubes and the turning point issue are resolved in practice.

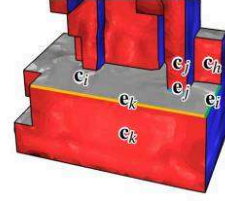


Figure 3: Charts c_h, c_k introduce a constraint with type I and charts c_j, c_k introduce a constraint with type II.

Remark There is no theoretical guarantee that our alternating normal-alignment deformation and normal-smooth deformation always resolve the above issues. However, in practice, several rounds of two types of deformations are enough to solve these issues. We list the number of rounds for all the experiments in Table 1.

3.5. Mesh Flattening

When the facet-orientation segmentation assures the existence of the PolyCube structure, we compute the PolyCube map by mesh flattening. The flattening algorithm contains two steps: (1) chart flattening; (2) fixed boundary mapping.

Chart flattening. Every chart is required to be exactly parallel to an axis plane, i.e., all the boundary vertices of a chart c_k share the same axis-coordinate value v_k . A naïve flattening method is to assign the average axis-coordinate m_k of all the vertices of c_k to v_k . However, inappropriate assignments may make the PolyCube structure physically invalid. For instance, two parallel charts might be coplanar and they introduce degenerate tetrahedrons. We introduce the following inequality constraints to avoid these cases robustly.

Constraint I. For any two charts c_k and c_h whose orientations are parallel, if they are connected by a PolyCube edge e_i , we impose

$$v_k - v_h \geq d_{k,h}.$$

Here $d_{k,h}$ is the length of e_i and we assume that the end point of e_i connected to c_k has a greater axis-coordinate value than the other end point of e_i .

Constraint II. For any two charts c_j and c_k whose orientations are parallel, if they are adjacent to a common chart c_i via the PolyCube edge e_j and e_k , e_j and e_k are not connected by a PolyCube edge, and the projected version of e_j and e_k on their axis direction overlap, we impose

$$v_k - v_j \geq d_{k,j}.$$

Here $d_{k,j}$ is the Euclidean distance between e_j and e_k , and we assume that the average axis-coordinate value of e_k is greater than the one of e_j .

Figure 3 illustrates the above two constraint types.

Under the above constraints, we minimize $\sum_i (v_i - m_i)^2$ to find appropriate axis-coordinate values v_i . The minimization can be done efficiently by a quadratic programming solver. In our implementation, we use the Mosek software [ApS15].

Fixed boundary mapping Once the PolyCube structure is determined, we can relocate the boundary vertices and interior vertices.

We treat the step as a fixed boundary mapping problem. For boundary vertices, we treat them as handles and their target positions are determined by substituting the known axis-coordinates into their current positions. We move PolyCube vertices, vertices on PolyCube edges, and vertices inside PolyCube charts to their desired positions sequentially, then improve the mapping by allowing the boundary vertices to move along its chart plane or its PolyCube edge. The optimization procedure is the same as [FLG15].

4. Experiments and Applications

We applied our PolyCube-map construction method to various complex volumetric meshes. Our experiments were conducted on a desktop computer with Intel Core i7-4770 processor and 16GB memory. We compared our method with state-of-the-art methods including l_1 -based construction [HJS*14], volumetric PolyCube deformation [GSZ11], PolyCut [LVS*13], deformation and voxel morphing [YZWL14], and singularity-restricted field [LLX*12]. We use the following quality metrics to measure the quality of the output of different methods.

Quality metric for PolyCube-map. The isometric distortion associated with tetrahedra $\mathbf{t}$ is defined as: $\delta_{\mathbf{t}} := \max(\sigma_{\max}, 1/\sigma_{\min})$. $\sigma_{\max}$ and $\sigma_{\min}$ are the maximal and minimal singular values of the linear transformation matrix $\mathbf{A}_{\mathbf{t}}$. $\delta_{\mathbf{t}}$ reaches its minimum 1 when there is no distortion. We report the maximum, average and standard deviation of the isometric distortions over all the tetrahedrons, denoted as $\delta_{\max}^{iso}$, $\delta_{\text{avg}}^{iso}$, $\delta_{\text{dev}}^{iso}$.

Quality metric for hexahedral meshes. We use the scaled Jacobian to measure the quality of hexahedral elements. The minimum, average and standard deviation of the scaled Jacobian over all the hexes are denoted as $J_{\min}$, J_{avg} and J_{dev} .

Table 1 summarizes the comparison results and our method is the fastest and generates better results than the others. In the following sections, we discuss these comparisons in detail.

4.1. PolyCube-maps

We demonstrate the strengths of our method on the following aspects: foldover-free and low distortion, controllable complexity, and computational efficiency.

Foldover-free and low distortion. The use of E_{iso} in our method assures that our result is foldover-free and large distortions are penalized. Figure 4 shows a comparison with [HJS*14] on the Elephant model. The result of [HJS*14] contains 592 foldovers because their method does not prevent foldovers in their PolyCube cleanup step, while our method produces a low-distortion volumetric mapping ($\delta_{\max}^{iso} = 11.714$) without foldovers. For other models like Kiss, Fertility, Bunny, Buste, Rocker Arm, Dancing Children, Dragon, the results of [HJS*14] also contain many foldovers (see the statistics of Table 1). For other deformation-based methods, they cannot get rid of foldovers as well (see Figure 9 of [GSZ11]) because their deformation energies [GSZ11, LVS*13] are not sensitive to foldovers and the parameterization [YZWL14] is not foldover-free. The occurrence of foldovers is the main obstacle for the all-hex meshing application especially when the expected edge length of the hex is small, and some integer grid points fall into the foldovers.

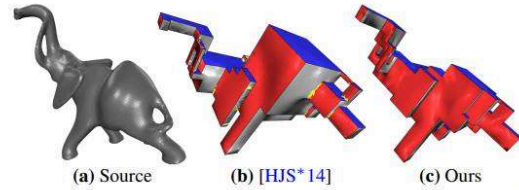


Figure 4: (a) The input Elephant model. (b) The result of [HJS*14] has 592 foldovers (colored in yellow). (c) Our foldover-free PolyCube mapping whose maximal isometric distortion is 11.714.

Controllable complexity. The Gaussian-smooth kernel size in our normal-smooth deformation provides a flexible way to control the complexity of the final PolyCube structure. Larger σ yields the result with fewer PolyCube corners. Figure 1(c),(d) and (e) show three results for the Buste model with different σ s. Figure 5 also demonstrates the controllable complexity of our method on other models.

Efficiency. Our method is at least one order magnitude faster than other deformation-based or optimization-based methods. Figure 6 shows a comparison with [HJS*14] using the similar number of tetrahedral elements as input. Our method is at least 30 times faster than [HJS*14]. We also compare with the method of [LVS*13] on the Kiss model. Since we have only the surface mesh provided by [LVS*13], we use this surface mesh to generate a tetrahedral mesh as our input which contains 442736 tetrahedrons. Our method took about 4 minutes to obtain the result, while [LVS*13] spent about 50 minutes to find the right segmentation. Table 1 shows more comparisons with other methods.

Practical robustness. We tested our method on 106 models to evaluate its practical robustness. Our method successfully computes the PolyCubes for all the models, and the results are available in the supplemental material.

4.2. All-Hex meshing

The PolyCube structure is useful for the all-hex meshing application. In our work, we modify the chart flattening procedure to get a hex-meshable PolyCube-map first, then construct the all-hex mesh and improve the mesh quality.

Chart flattening. For all-hex meshing, the axis coordinate v_i of each chart c_i should be an integer. Assume that the expected hexahedral edge length is g_H , we modify the constraints and the objective function of Section 3.5 as follows: set m_i as the nearest integer to $\frac{m_i}{g_H}$, set the right side d of every inequality equation as $\max(\text{round}(\frac{d}{g_H}), 1)$. We then solve the quadratic programming problem and round each v_i to its nearest integer. Note that the rounding results do violate the constraints because the distance difference between the two related charts is greater than or equal to 1. Once v_i s are known, we scale the deformed mesh by $\frac{1}{g_H}$ and do fixed boundary mapping. Finally, an all-hex mesh can be obtained by projecting the grid points inside the PolyCube back to the input volume by their barycenter coordinates.

All-Hex mesh improvement. Similar to the work [GSZ11, Mar09], we first insert a padding layer on the boundary of $\mathcal{M}^H$ to embed the corners of the PolyCube inside $\mathcal{M}^H$. Notice that the scaled Jacobian values of some hex elements are not positive due to the

X. Fu & C. Bai & Y. Liu / Efficient Volumetric PolyCube-Map Construction

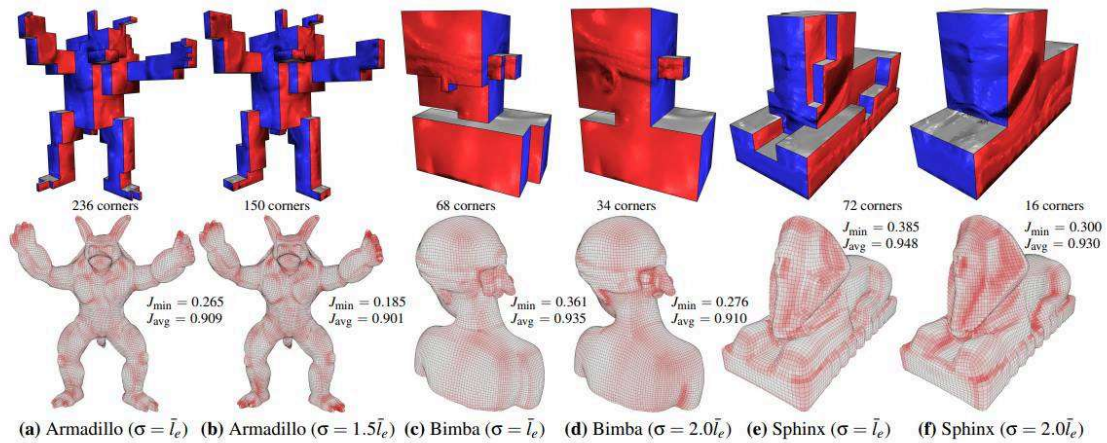


Figure 5: PolyCube-maps by different σ s on three models. The first row shows the PolyCube structure and the second row shows the corresponding all-hex meshing results. Hexes are colored by their scaled Jacobian values. The hex meshes from the PolyCubes with more corners have better quality.

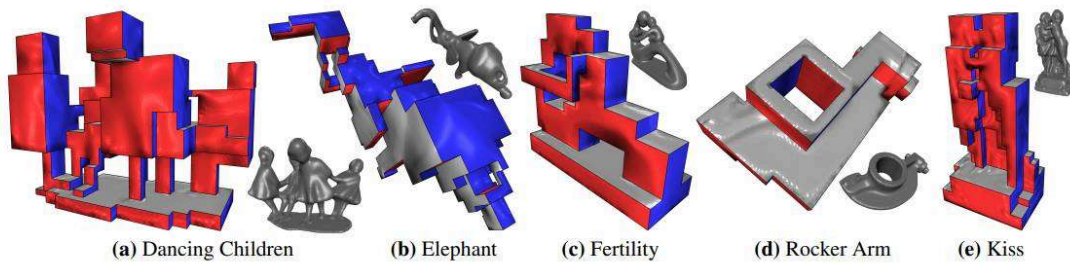


Figure 6: From (a) to (d), we generate PolyCubes using a similar number of tetrahedral elements with [HJS*14] and our method took 0.92, 1.15, 0.46, 0.35 minutes, respectively, while the method of [HJS*14] took about a half hour for each model. We also compare with [LVS*13] on the Kiss model. Our method only spent about 4 minutes, while their method cost about 50 minutes as reported in [LVS*13].

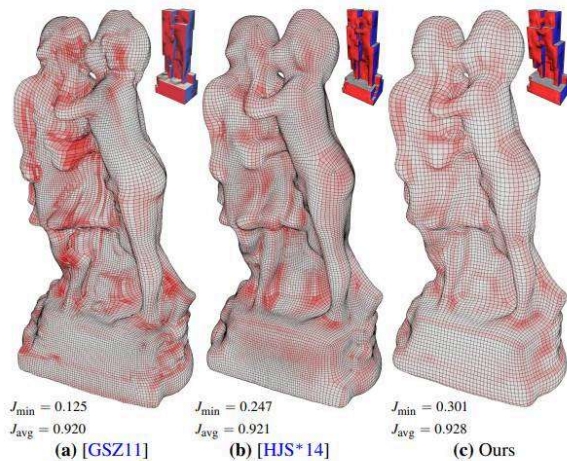


Figure 7: Comparison on the Kiss model.

back projection, i.e., some hex elements are tangled. We employ the method of [ERM*03] to improve the quality of the all-hex mesh and untangle the mesh simultaneously.

Figure 7 shows a comparison with [GSZ11] and [HJS*14] on the Kiss model. The number of hexahedral elements of [GSZ11], [HJS*14] and ours are 215320, 219009 and 57451. The minimal scaled Jacobian and average scaled Jacobian are (0.125, 0.920), (0.247, 0.921) and (0.301, 0.928), respectively. We can see that our method achieves the best quality with the least number of hexahedral elements. Figure 8 shows the comparison of other two models with other PolyCube-based all-hex meshing methods. The quality of our results is still the best. More statistics of the comparison is given in Table 1.

Figure 9 shows all-hex meshing using different expected hexahedral edge lengths. g_{HS} in Figure 9(a) and Figure 9(b) are set as 1.6 and 0.9 times the average boundary edge length.

Figure 10 compares our method with the frame-driven all-hex meshing method [LLX*12]. Our results have better quality but contain more singularity points on the boundary, and the size of hexes is not uniform due to the nature of the PolyCube structure.

Figure 12 shows more results of our PolyCube maps and all-hex meshing.

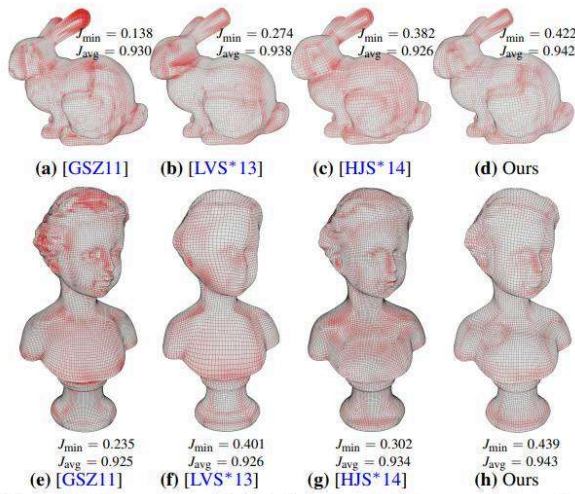


Figure 8: Comparison with PolyCube based all-hex meshing methods. We color-code hexes by their scaled Jacobian values, with white being the optimal. Our method achieves the best quality.

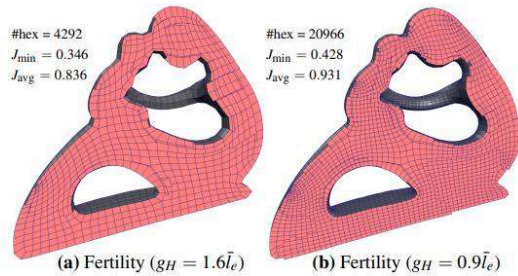


Figure 9: Two all-hex meshes are generated from the same PolyCube structure and with different number of elements for Fertility model. (a) and (b) are cut-views of two all-hex meshes with 4292 and 20966 elements, respectively.

5. Conclusion and Discussion

We present a novel and automatic method to compute foldover-free and complexity-controllable PolyCube-maps. Our method is much faster than existing methods and has better quality both in PolyCube-maps and all-hex meshing. We also note a few limitations and would like to address them in future work.

Theoretical guarantee. Currently, only voxel-based PolyCube construction methods can assure the existence of the PolyCube structure, but the result quality of these methods might be very poor near the boundary of the shape. The optimization-based or deformation-based methods, like ours, can usually generate high-quality results but have no theoretical guarantee that any invalid PolyCube topology is always revisable and all the turning points can be eliminated.

Sharp features. It is known that PolyCubes hardly preserve sharp features of models. Figure 11 shows that our method cannot align the sharp features of the models to the PolyCube edge well.

Gaussian kernel size. The Gaussian kernel size affects the complexity of the final PolyCube. If it is larger than the size of surface

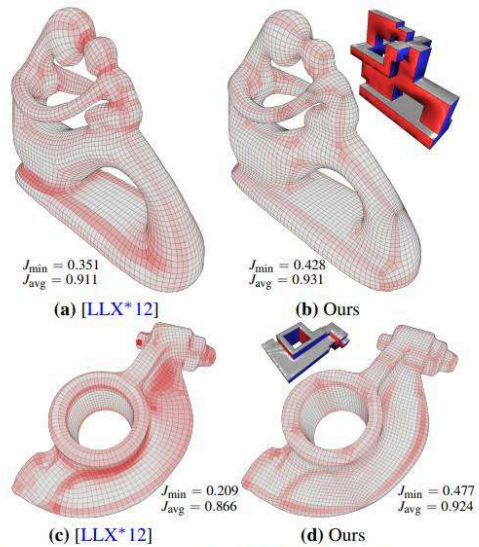


Figure 10: Comparison with [LLX*12] on all-hex meshing.

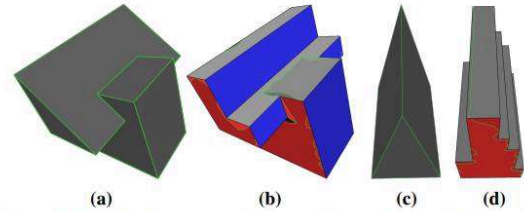


Figure 11: PolyCube construction for the meshes with sharp features (green lines). (b) and (d) are the PolyCubes of (a) and (c), respectively. The sharp features of input mesh are not mapped to the PolyCube edges perfectly.

features where the user wants to preserve, these features might be smoothed out. So a spatially-varying kernel size with respect to the importance of features is essential for achieving the desired PolyCube structure. In future, we would like to incorporate the user interaction to identify important features.

Topological conditions for PolyCube. In our method, we use the sufficient topological conditions for PolyCube. It is known that these conditions are not necessary conditions [GSZ11, LVS*13]. So seeking a weaker sufficient condition would help us find the PolyCube structure more easily.

Acknowledgements

Mesh models are courtesy of the Aim@Shape Repository, the Stanford 3D Scanning Repository and the dataset of [MPZ14].

References

- [AL13] AIGERMAN N., LIPMAN Y.: *Injective and bounded distortion mappings in 3D*. *ACM Trans. Graph. (SIGGRAPH)* 32, 4 (2013), 106:1–106:14. 2
- [APs15] APS M.: *The MOSEK optimization toolbox*, 2015. 5

X. Fu & C. Bai & Y. Liu / Efficient Volumetric PolyCube-Map Construction

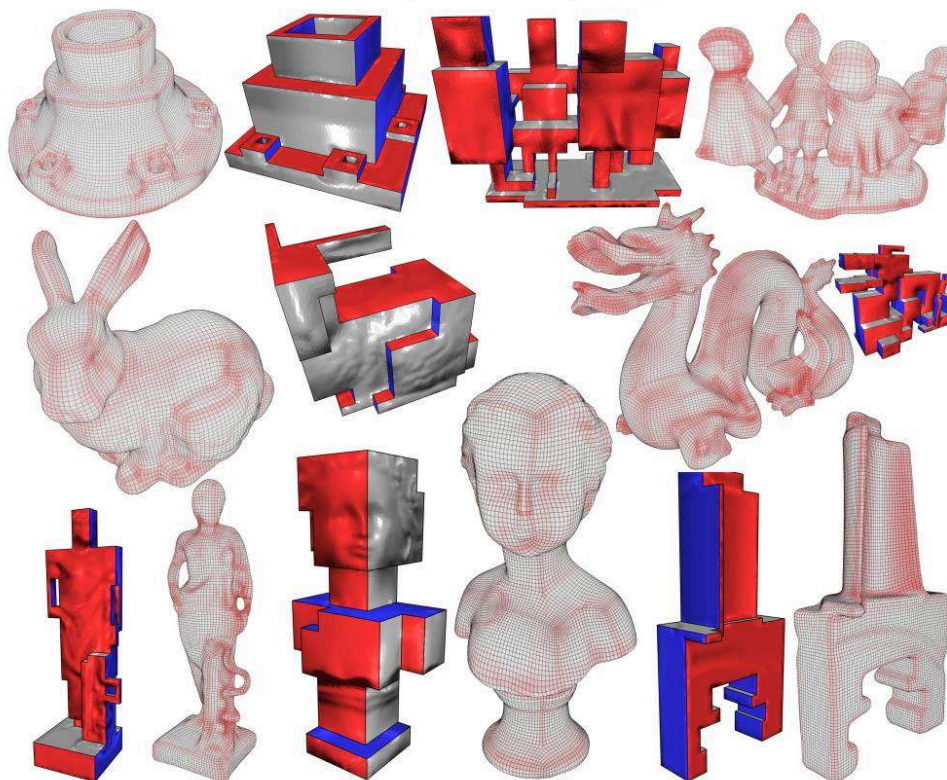


Figure 12: Gallery of our polycube maps and all-hex meshing results.

- [CL10] CHANG C.-C., LIN C.-Y.: *Texture tiling on 3D models using automatic PolyCube-maps and wang tiles*. *Journal of Information Science and Engineering* 26, 1 (2010), 291–305. 1
- [EM10] EPPSTEIN D., MUMFORD E.: *Steinitz Theorems for orthogonal polyhedra*. In *Symp. on Comp. Geom.* (2010), pp. 429–438. 4
- [ERM*03] ESCOBAR J., RODRÍGUEZ E., MONTENEGRO R., MONTERO G., GONZÁLEZ-YUSTE J.: *Simultaneous untangling and smoothing of tetrahedral meshes*. *Comput. Methods Appl. Mech. Engrg.* 192 (2003), 2775–2787. 7
- [FLG15] FU X.-M., LIU Y., GUO B.: *Computing locally injective mappings by advanced MIPS*. *ACM Trans. Graph. (SIGGRAPH)* 34, 4 (2015), 71:1–71:12. 2, 3, 4, 6
- [Flo03] FLOATER M. S.: *Mean value coordinates*. *Comput. Aided Geom. Des.* 20, 1 (2003), 19–27. 2
- [FXBH16] FANG X., XU W., BAO H., HUANG J.: *All-hex meshing using closed-form induced Polycube*. *ACM Trans. Graph. (SIGGRAPH)* 35, 4 (2016), 124:1–124:9. 1
- [GSZ11] GREGSON J., SHEFFER A., ZHANG E.: *All-Hex mesh generation via volumetric PolyCube deformation*. *Comput. Graph. Forum (SGP)* 30 (2011), 1407–1416. 1, 2, 3, 4, 6, 7, 8, 10
- [HJS*14] HUANG J., JIANG T., SHI Z., TONG Y., BAO H., DESBRUN M.: *l_1 -based construction of PolyCube maps from complex shapes*. *ACM Trans. Graph.* 33, 3 (2014), 25:1–25:11. 1, 2, 4, 6, 7, 8, 10
- [HWFQ09] HE Y., WANG H., FU C.-W., QIN H.: *A divide-and-conquer approach for automatic PolyCube map construction*. *Computers & Graphics* 33, 3 (2009), 369–380. 2
- [HXH10] HAN S., XIA J., HE Y.: *Hexahedral shell mesh construction via volumetric Polycube map*. In *ACM Symp. on Solid and Physical Modeling* (2010), pp. 127–136. 2
- [KABL15] KOVALSKY S. Z., AIGERMAN N., BASRI R., LIPMAN Y.: *Large-scale bounded distortion mappings*. *ACM Trans. Graph. (SIGGRAPH ASIA)* 34, 6 (2015), 191:1–191:10. 2
- [LGW*07] LI X., GUO X., WANG H., HE Y., GU X., QIN H.: *Harmonic volumetric mapping for solid modeling applications*. In *ACM Symp. on Solid and Physical Modeling* (2007), pp. 109–120. 2
- [LJFW08] LIN J., JIN X., FAN Z., WANG C. C.: *Automatic PolyCube-maps*. In *Advances in Geometric Modeling and Processing* (2008), pp. 3–16. 2
- [LLX*12] LI Y., LIU Y., XU W., WANG W., GUO B.: *All-hex meshing using singularity-restricted field*. *ACM Trans. Graph. (SIGGRAPH ASIA)* 31, 6 (2012), 177:1–177:11. 6, 7, 8, 10
- [LVS*13] LIVESU M., VINING N., SHEFFER A., GREGSON J., SCATENI R.: *Polycut: monotone graph-cuts for PolyCube base-complex construction*. *ACM Trans. Graph. (SIGGRAPH ASIA)* 32, 6 (2013), 171:1–171:12. 1, 2, 4, 6, 7, 8, 10
- [Mar09] MARÉCHAL L.: *Advances in octree-based all-hexahedral mesh generation: handling sharp features*. In *Int. Meshing Roundtable* (2009), pp. 65–84. 6
- [MPZ14] MYLES A., PIETRONI N., ZORIN D.: *Robust field-aligned global parametrization*. *ACM Trans. Graph. (SIGGRAPH)* 33, 4 (2014), 135:1–135:14. 8
- [SABL14] SHAHAR K., AIGERMAN N., BASRI R., LIPMAN Y.: *Controlling singular values with semidefinite programming*. *ACM Trans. Graph. (SIGGRAPH)* 33, 4 (2014), 68:1–68:13. 2

X. Fu & C. Bai & Y. Liu / Efficient Volumetric PolyCube-Map Construction

Model	#vert/#tet	$\sigma(\times \bar{l}_e)$	$\delta_{\max}^{\text{iso}}/\delta_{\text{avg}}^{\text{iso}}/\delta_{\text{dev}}^{\text{iso}}$	#round	time (m)	#corner	#vert/#hex	$J_{\min}/J_{\text{avg}}/J_{\text{dev}}$	$D_H(\times 10^{-3})$
Kiss ([GSZ11])	—	—	—	—	—	153	230546/215320	0.125/0.920/0.086	10.2
Kiss ([HJS*14])	26581/102761	—	1099 foldovers	—	37.3	170	236160/219009	0.247/0.921/0.073	14.8
Kiss (Figure 7(c))	14157/54025	0.75	10.743/1.527/0.555	2	0.26	146	63997/57451	0.301/0.928/0.072	2.85
Kiss (Figure 6(e))	103923/442736	1.6	10.347/1.468/0.491	5	3.84	200	—	—	—
Armadillo (Figure 5(a))	21599/81114	1.0	6.660/1.532/0.405	3	0.46	236	87303/78376	0.265/0.909/0.080	4.59
Armadillo (Figure 5(b))	21599/81114	1.5	7.164/1.663/0.520	4	0.64	150	102757/92869	0.185/0.901/0.093	5.27
Bimba (Figure 5(c))	15600/61179	1.0	4.624/1.426/0.366	3	0.40	68	73104/67039	0.361/0.935/0.068	3.69
Bimba (Figure 5(d))	15600/61179	2.0	6.001/1.477/0.414	3	0.44	34	70994/65344	0.276/0.910/0.081	3.32
Sphinx (Figure 5(e))	13188/48623	1.0	7.690/1.451/0.412	1	0.23	72	46544/42149	0.385/0.948/0.072	7.42
Sphinx (Figure 5(f))	13188/48623	2.0	4.774/1.513/0.342	3	0.48	16	69634/64398	0.300/0.930/0.091	4.80
Fertility ([GSZ11])	—	—	—	—	—	96	23653/19870	0.196/0.911/0.100	6.96
Fertility ([LVS*13])	—	—	—	—	—	94	61147/53702	0.259/0.872/0.108	4.33
Fertility ([HJS*14])	24652/90486	—	1109 foldovers	—	28.6	95	21285/17910	0.312/0.911/0.087	12.3
Fertility ([YZWL14])	—	—	—	—	13.6	146	38034/31333	-0.991/0.845/0.199	5.34
Fertility ([LLX*12])	—	—	—	—	—	—	16531/13584	0.351/0.911/0.088	13.6
Fertility (Figure 9(a))	8657/31283	1.0	10.002/1.766/0.563	2	0.16	104	5447/4292	0.346/0.836/0.125	8.86
Fertility (Figure 9(b), 10(b))	8657/31283	1.0	10.625/1.746/0.587	2	0.16	104	24468/20700	0.428/0.931/0.069	7.45
Fertility (Figure 6(c))	24634/94023	2.0	8.322/1.585/0.526	2	0.56	108	—	—	—
Bunny ([GSZ11])	—	—	—	—	—	90	88607/81637	0.138/0.930/0.084	4.01
Bunny ([LVS*13])	—	—	—	—	—	64	80602/74084	0.274/0.938/0.076	9.23
Bunny ([HJS*14])	40107/164419	—	947 foldovers	—	130	76	41749/37734	0.382/0.926/0.080	7.30
Bunny ([YZWL14])	—	—	—	—	14.7	84	47549/42543	-0.948/0.900/0.177	18.8
Bunny ([LLX*12])	—	—	—	—	—	—	141209/133632	0.293/0.940/0.079	24.9
Bunny (Figure 8(d), 12)	16274/63379	0.75	7.061/1.588/0.388	3	0.46	84	65603/59841	0.422/0.942/0.062	4.28
Buste ([GSZ11])	—	—	—	—	—	88	206437/193664	0.235/0.925/0.075	13.2
Buste ([LVS*13])	—	—	—	—	—	60	61308/55607	0.401/0.926/0.067	6.64
Buste ([HJS*14])	23704/93806	—	1198 foldovers	—	28.4	93	222372/208928	0.302/0.934/0.062	7.66
Buste (Figure 8(h), 12)	24985/98165	1.3	8.445/1.502/0.415	2	0.51	82	86595/79294	0.439/0.943/0.057	3.24
Buste (Figure 1(c))	24985/98165	1.0	9.201/1.495/0.478	2	0.46	128	—	—	—
Buste (Figure 1(d))	24985/98165	1.5	7.840/1.532/0.398	3	0.58	72	—	—	—
Buste (Figure 1(e))	24985/98165	2.0	5.874/1.574/0.374	3	0.57	36	—	—	—
Rocker arm ([LVS*13])	—	—	—	—	—	62	63363/56667	0.370/0.890/0.090	3.12
Rocker arm ([HJS*14])	26532/123813	—	190 foldovers	—	39.5	88	28397/24346	0.439/0.920/0.076	8.37
Rocker arm ([YZWL14])	—	—	—	—	16.6	80	10078/7874	-0.506/0.869/0.210	7.08
Rocker arm ([LLX*12])	—	—	—	—	—	—	12751/10600	0.209/0.866/0.115	17.1
Rocker arm (Figure 10(d))	8952/33417	0.8	5.874/1.520/0.377	1	0.15	66	27141/23421	0.477/0.924/0.074	5.27
Rocker arm (Figure 6(d))	27960/110776	1.6	6.731/1.467/0.383	1	0.35	66	—	—	—
Carter ([LVS*13])	—	—	—	—	—	128	76512/65214	0.250/0.895/0.110	9.37
Carter (Figure 12)	24201/92862	1.2	7.759/1.696/0.598	3	0.46	140	70963/60412	0.366/0.917/0.094	9.10
Elephant ([HJS*14])	27103/131979	—	592 foldovers	—	23.4	160	186675/171657	0.221/0.878/0.114	8.06
Elephant (Figure 1(a, b), 4(c))	13188/48623	1.5	11.714/1.835/0.646	2	0.29	176	53218/46994	0.231/0.887/0.100	3.35
Elephant (Figure 6(b))	35450/136774	1.6	8.331/1.532/0.467	4	1.15	282	—	—	—
Dancing Children ([HJS*14])	25950/114170	—	1030 foldovers	—	27.7	210	41645/35293	0.143/0.870/0.106	23.9
Dancing Children (Figure 12)	16433/59769	1.0	7.097/1.684/0.467	2	0.27	216	36353/30691	0.251/0.878/0.099	5.11
Dancing Children (Figure 6(a))	32262/121516	2.0	5.841/1.537/0.431	4	0.92	292	—	—	—
Dragon ([HJS*14])	26779/104261	—	1532 foldovers	—	28.1	315	131367/117725	0.150/0.857/0.145	23.1
Dragon (Figure 12)	29114/109823	1.2	10.184/1.739/0.538	3	0.76	210	118610/106244	0.265/0.862/0.106	16.1
Blade (Figure 12)	20331/79413	1.0	7.231/1.404/0.364	1	0.28	68	75885/68383	0.387/0.920/0.100	3.50
Greek Sculpture (Figure 12)	17434/65659	1.5	9.954/1.651/0.561	5	0.92	108	41299/36184	0.250/0.901/0.107	7.79

Table 1: Statistics and timings for PolyCube construction and all-hex meshing. We report the number of vertices and tetrahedrons of the input mesh (“#vert/#tet”), the Gaussian kernel size σ ($\bar{l}_e$ is the average boundary edge length), the isometric distortion metric, the number of optimization rounds (“#round”) in our algorithm, the computational time in minutes, the number of corners (“#corner”) in the resulting PolyCube. We also report the statistics of the all-hex meshing results: number of vertices and hexes of the all-hex mesh (“#vert/#hex”), the scaled Jacobian metric, the maximum Hausdorff distance D_H (scaled by the diagonal length of the bounding box of the input) between the boundary triangle surface and the boundary surface of the all-hex mesh. The results of [GSZ11], [LVS*13], [HJS*14], [YZWL14] and [LLX*12] are obtained from the authors’ websites. We label the unavailable data with “-”.

[Si15] SI H.: TetGen, a Delaunay-Based quality tetrahedral mesh generator. *ACM Trans. Math. Softw.* 41, 2 (2015), 11:1–11:36. 3

[SKPSH13] SCHÜLLER C., KAVAN L., PANOZZO D., SORKINE-HORNUNG O.: Locally injective mappings. *Comput. Graph. Forum (SGP)* 32, 5 (2013), 125–135. 2

[SR15] SOKOLOV D., RAY N.: *Fixing normal constraints for generation of PolyCubes*. Tech. rep., LORIA, 2015. 2

[THCM04] TARINI M., HORMANN K., CIGNONI P., MONTANI C.: PolyCube-Maps. *ACM Trans. Graph. (SIGGRAPH)* 23, 3 (2004), 853–860. 1, 2

[XGH*11] XIA J., GARCIA I., HE Y., XIN S.-Q., PATOW G.: Editable polycube map for GPU-based subdivision surfaces. In *Symp. on Interactive 3D Graphics and Games* (2011), pp. 151–158. 1

[YL08] YAO C.-Y., LEE T.-Y.: Adaptive geometry image. *IEEE. T. Vis. Comput. Gr.* 14, 4 (2008), 948–960. 1

[YZWL14] YU W., ZHANG K., WAN S., LI X.: Optimizing Polycube domain construction for hexahedral remeshing. *Computer Aided Design* 46 (2014), 58–68. 2, 6, 10